Cẩm nang cải thiện pipeline S2S trên QCS8550: giảm trễ, giảm dung lượng, nâng chất lượng (không train lại)

Ngày soạn: 10/11/2026. Phạm vi: pipeline `s2s_edge` (ASR → làm sạch văn bản → MT → TTS) cho vi↔en, vi↔zh, vi↔ko trên HSPTEK AIBOX 8550 (QCS8550, Hexagon v73).

0. Cách đọc tài liệu này

Mỗi kỹ thuật có nhãn độ tin cậy. Đọc nhãn trước khi tin con số.

Nhãn	Nghĩa
[Đã đọc]	Tôi đã tải bài báo hoặc tài liệu, trích chữ và đọc đúng đoạn nêu ở đây. Có ghi mục, bảng, hình.
[Chỉ tóm tắt]	Chỉ thấy kết quả tìm kiếm hoặc tóm tắt, chưa đọc nguyên văn. Không dùng làm căn cứ chính.
[Đo của ta]	Số liệu từ thử nghiệm của chính nhóm (CPU hoặc profile chip). Có ghi nguồn trong repo.
[Giả thuyết]	Suy luận kỹ thuật hợp lý nhưng chưa ai đo trên pipeline này. Cần thử trước khi tin.

Ba điều cần nhớ khi dùng các bài báo:

- Hầu hết bài báo đo trên GPU (V100, A40, RTX 4090). Con số tuyệt đối không chuyển sang QCS8550. Chỉ chiều hướng (tăng hay giảm) là đáng tham khảo.
- Hầu hết đo trên en↔de/es/it/fr, không phải tiếng Việt, Hàn, Trung.
- Mọi kỹ thuật dưới đây phải được kiểm bằng `tools/eval_btc.py` (chrF/BLEU, WER/CER, "heard", độ trễ) trước khi đưa vào gói nộp. Bộ BTC chỉ có 100 đến 180 clip mỗi chiều và nhãn có nhiễu, nên chênh lệch nhỏ (dưới khoảng 1 chrF) không kết luận được.
- Năm xuất bản lấy theo mã arXiv. Tôi không ghi tên hội nghị vì chưa kiểm từng bài.

0.1 Điểm xuất phát (số của ta)

Đơn vị dung lượng là MiB (kích thước tệp chia cho 1024^2).

Mục	Số liệu	Nguồn
Dung lượng đồ thị biên dịch	ASR 603, MT 2.379 (enc 611 + dec 884 + decB 884), TTS 1.004; tổng khoảng 3.990	gói `models/`
Một chiều dịch nạp	khoảng 2,2 GB	`aibox8550_report.md`
Zipformer-vi CTC fp16, cửa sổ 15 s	163 ms (profile QCS8550), 174 ms (đo trước đó trên IQ-9075)	[Đo của ta] `profile.json`
NLLB encoder S=64	7,4 ms (IQ-9075)	[Đo của ta]
NLLB decoder một bước, greedy	16,1 ms (IQ-9075)	[Đo của ta]
NLLB decoder beam 4 (decB), một bước	30,8 ms (QCS8550)	[Đo của ta] `profile.json`

Mục	Số liệu	Nguồn
Ước tính chip cho một câu ngắn	ASR 0,16 đến 0,33 s, MT 0,4 đến 0,8 s, TTS 0,5 đến 0,75 s	ước tính, chưa đo end-to-end
Từ vựng NLLB trong đồ thị	đã cắt còn 138.974 trên 256.206 token (54 %); embedding fp16 khoảng 271 MiB	`keep_all.json`, `nllb_static.py`

Hai điều rút ra: độ trễ nằm ở **MT và TTS**; và **embedding đầu ra vẫn chiếm khoảng 31 % dung lượng decoder** (271 trên 884 MiB).

PHẦN 1. Giảm độ trễ và giảm dung lượng mô hình

1.1 Cắt từ vựng đầu ra sâu hơn

Kỹ thuật. Mỗi bước giải mã, đồ thị decoder nhân vector ẩn với ma trận embedding (dùng chung làm lớp đầu ra) để ra điểm cho từng token. Ma trận càng lớn càng tốn băng thông bộ nhớ. Bỏ các token không bao giờ dùng thì kết quả không đổi, còn ma trận nhỏ đi.

Tài liệu. [Đã đọc] Ushio, Zhou, Camacho-Collados, *An Efficient Multilingual Language Model Compression through Vocabulary Trimming* (2023), arXiv 2305.15020.

- Mục 3 và Hình 2: giữ lại các token xuất hiện trong ngữ liệu của ngôn ngữ đích, xóa phần còn lại; có biến thể chỉ giữ top-n token thường gặp để nhỏ hơn nữa.
- Hình 1 và phần Giới thiệu: ma trận embedding có thể chiếm hơn 80 % tham số của một số mô hình đa ngữ.
- Phần Giới thiệu: giữ được hiệu năng gốc với dưới 40 % tham số trên các tác vụ thử.
- Mục 3: biến thể cắt sau khi fine-tune không cần huấn luyện thêm.
- **Giới hạn của bài.** Các tác vụ thử là hỏi đáp, sinh câu hỏi, cảm xúc, suy luận ngôn ngữ, trên bốn mô hình đa ngữ (gồm mT5, XLM-R, mBART). **Không có dịch máy.** Phần Limitations nói chưa thử ngôn ngữ ít dữ liệu.

Của ta. Đồ thị đã cắt một lần: `export_nllb_split2.py` gọi `NLLBStatic(..., keep_ids=keep, half_emb=True)` với `keep_all.json` (138.974 id). Tôi suy ra con số này lớn vì phải phủ cả bốn ngôn ngữ, mà tiếng Trung và Hàn có nhiều token; chưa kiểm cách tệp đó được tạo. Cắt thêm có hai hướng:

- **Một decoder cho mỗi ngôn ngữ đích.** Lớp đầu ra chỉ cần token của ngôn ngữ đích (ví dụ decoder đích tiếng Việt chỉ giữ token tiếng Việt và ký hiệu đặc biệt). Cần đếm số token mỗi ngôn ngữ để biết kích thước thật.
- **Giữ top-n theo tần suất** (biến thể ở Mục 3 của bài), n chọn theo ngữ liệu miền doanh nghiệp.

Tác động dự kiến. [Giả thuyết] Nếu một bước decoder bị giới hạn bởi băng thông bộ nhớ (chưa kiểm), giảm số byte đọc mỗi bước sẽ giảm thời gian bước gần tỉ lệ. Bước 16,1 ms mà phải đọc khoảng 0,9 GB trọng số là dấu hiệu đáng nghi, nhưng cần profile để xác nhận. Tôi chưa tra băng thông của QCS8550 nên không tính ra con số mong đợi.

Rủi ro. Token ngoài tập giữ bị map sang UNK; câu có tên riêng lạ có thể sinh sai. Đã có `step3_prune_parity.py` và `step3b_parity_allscripts.py` để kiểm tính tương đương.

Cách kiểm trên laptop. Giới hạn logits theo danh sách id mới trong host beam search trên CPU, so chrF/BLEU trên BTC với bản hiện tại. Kết quả phải gần như trùng, trừ khi câu dùng token bị cắt.

1.2 Lượng tử hóa trọng số (int8 hoặc W8A16)

Kỹ thuật. Lưu trọng số 8 bit thay vì 16 bit. Dung lượng gần giảm một nửa. Nếu bước giải mã bị giới hạn bởi băng thông, thời gian bước cũng giảm.

Bằng chứng của ta.

- **[Đo của ta]** `step4.md` mục 2: NLLB-600M qua CTranslate2 int8, 2.300 MB → 594 MB; BLEU trên FLORES cả 6 chiều không giảm (vi→en 33,81→34,59; en→vi 29,67→29,31; vi→zh 20,45→21,25; zh→vi 21,07→24,01; vi→ko 8,05→8,59; ko→vi 18,60→22,65). Báo cáo ghi rõ chênh nhỏ nằm trong sai số của beam search.
- **Giới hạn phép đo đó.** Đây là lượng tử hóa **trọng số** của CTranslate2 trên CPU. Chưa chắc giống lượng tử hóa của Qualcomm (có thêm lượng tử hóa kích hoạt).
- **[Đo của ta]** `step4.md` mục 2 và 3b: SenseVoice-Small với int8 động của ONNX Runtime làm CER tiếng Trung tăng 2,3 %→9,8 %, tiếng Hàn 4,5 %→9,5 % (mẫu chỉ 5 câu mỗi ngôn ngữ). Supertonic: lượng tử hóa `vocoder.onnx` làm âm thanh vỡ; giữ vocoder fp32 thì ba thành phần còn lại ổn (398 → 178 MB).
- **[Đo của ta]** Zipformer w8a16 trên chip kém (đã ghi từ phiên trước). Encoder SenseVoice trong gói hiện dùng w8a16 nhưng chất lượng riêng của nó chưa đo tách khỏi phần còn lại.

Tài liệu.

- **[Đã đọc]** AIMET, Quantization workflow: trình tự khuyến nghị là (1) thử FP16, nếu đủ nhanh thì không cần lượng tử hóa; (2) kiểm W16A16 với QuantSim (Step 2); (3) hạ xuống W8A16 (Step 3); (4) khôi phục độ chính xác bằng PTQ, mixed precision hoặc QAT, "nên bắt đầu từ PTQ" (Step 4). Trang này không nói về số mẫu hiệu chuẩn.
- **[Đã đọc]** AI Hub Workbench, Quantize examples:
- Mục "Quantization options": runtime QNN dùng trọng số int8 và kích hoạt int8 hoặc int16. Trang ghi **không runtime nào hỗ trợ mixed precision**, và int4 không có trong bảng.
- Mục "Quantize an ONNX model": khuyến nghị 500 đến 1000 mẫu hiệu chuẩn.
- Phần mở đầu "Quantize jobs": mô hình lượng tử hóa có thể nhanh hơn tới 3 lần.
- Ví dụ duy nhất là MobileNetV2. **Không có ví dụ NLP, transformer hay ASR.** Trang nói tính năng còn đang hoàn thiện cho nhiều loại mô hình.
- **[Đã đọc]** CTranslate2, Quantization: int8 chỉ lượng tử hóa trọng số của lớp embedding và linear theo tỉ lệ đối xứng; bảng kích thước cho Transformer base: float32 364 MB, float16 182 MB, int8_float16 95 MB.

Kết luận. Ba đường khác nhau, không được lẫn:

Đường	Chạy trên NPU?	Bằng chứng chất lượng
CTranslate2 int8 (CPU)	Không	Có, FLORES 6 chiều (cho thấy trọng số int8 của NLLB không hại)
AI Hub Workbench quantize	Có	Chưa có cho NLLB; trang không có ví dụ transformer
AIMET trực tiếp, rồi nạp vào AI Hub	Có	Chưa có

Nếu cần giảm dung lượng MT: thử W8A16 cho trọng số encoder và decoder, giữ kích hoạt 16 bit, và đo chrF bằng mô phỏng (QuantSim) trên CPU **trước** khi tốn thời gian chạy job trên AI Hub. Tránh nén vocoder TTS và encoder SenseVoice nếu chưa đo.

1.3 Chia sẻ trọng số giữa các graph

Hiện trạng. `mt_dec_nllb_s64.bin` và `mt_decB_nllb_s64.bin` (mỗi tệp 884 MiB) chứa gần như cùng trọng số, chỉ khác cách xử lý beam.

Tài liệu. [Đã đọc] QNN HTP backend, bảng Context Config Options: tùy chọn `QNN_HTP_CONTEXT_CONFIG_OPTION_WEIGHT_SHARING_ENABLED` cho phép nhiều graph (tối đa 64) dùng chung trọng số trong một context binary. Giới hạn ghi ở chú thích ngay sau: **chuẩn bị offline chỉ trên Linux x86**, chỉ từ Hexagon v73 trở lên, chỉ trong một PD, và binary đã tạo trước đó không tự hưởng lợi.

Của ta. v73 thỏa điều kiện. Nhưng bin của ta tạo qua **AI Hub link job**, chưa rõ job có bật tùy chọn này hay không. Chưa kiểm. Hai việc cần làm:

- Tra hoặc hỏi AI Hub có cho cấu hình weight sharing khi link nhiều graph vào một context không.
- Nếu không, phương án hạ cấp: chỉ dùng greedy (bỏ decB, mất beam) hoặc chỉ dùng decB (mất tốc độ greedy). Quyết định bằng chrF trên BTC.

Tiết kiệm dự kiến. [Giả thuyết] Tối đa khoảng 0,9 GB nếu hai decoder dùng chung trọng số. Cũng mở đường cho nhiều graph theo kích thước ở mục 1.5 mà không nhân đôi dung lượng.

1.4 Chế độ hiệu năng HTP và đo từng lớp

Tài liệu. [Đã đọc] QNN HTP backend, mục "QNN HTP Backend Extensions": khóa `perf_profile` (trong `devices` → `cores`) nhận các giá trị `low_balanced`, `balanced`, `high_performance`, `sustained_high_performance`, `burst`, các mức tiết kiệm điện và `system_settings`. Trang nói `burst` có tần số cao nhất và giữ phiếu bầu cao liên tục; `high_performance` **không giữ** phiếu qua nhiều lần suy luận; `burst` và `sustained_high_performance` dùng bộ đếm thời gian trong lúc chạy.

Đọc thêm Profile your AI model using QAIRT SDK và QNN Benchmarking **[Chỉ tóm tắt]** (tôi chưa đọc hết hai trang này).

Của ta. Gói đã đặt `PerfProfile.BURST` qua `qai_appbuilder`. Chưa xác nhận trên máy thật rằng chế độ có hiệu lực, và với tải ngắt quãng (mỗi câu một lần, có khoảng nghỉ) thì việc chip hạ xung giữa các câu ảnh hưởng thế nào chưa biết. Việc đầu tiên trên máy thật: chạy `session0.sh` ở hai chế độ (BURST và mặc định) và so thời gian.

1.5 Đồ thị nhiều kích thước thay cho một cửa sổ cố định

Hiện trạng. Đồ thị NPU có hình dạng cố định. ASR Zipformer luôn xử lý cửa sổ 15 s (khoảng 170 ms) dù clip chỉ 2 s. TTS dùng cửa sổ cố định nên câu ngắn vẫn tốn 0,5 đến 0,75 s. MT có `S=64` và `Tm=96`.

Kỹ thuật. [Giả thuyết] Biên dịch nhiều cửa sổ (ví dụ ASR 5 s, 10 s, 15 s; TTS nhỏ, vừa, lớn) và chọn cửa sổ nhỏ nhất đủ chứa đầu vào. Chi phí tính toán gần tỉ lệ độ dài nên câu ngắn nhanh hơn. Đây là kỹ thuật phần mềm; **tôi không có bài báo nào** nói về nó cho thiết bị này.

Chi phí. Mỗi graph thêm làm tăng dung lượng. Nếu mục 1.3 dùng được thì chi phí gần bằng 0. Nếu không, bắt đầu với ASR (graph Zipformer 335 MiB) vì TTS và MT lớn hơn nhiều.

Việc cần đo trước. Phân bố độ dài clip BTC (giây). Nếu hầu hết clip dài hơn 8 s thì lợi ích rất nhỏ.

1.6 Phát âm thanh sớm và gối đầu bên trong một câu

Tài liệu. [Đã đọc] Liu, Wang, Gong, Ma, Tang, Pino, *From Start to Finish: Latency Reduction Strategies for Incremental Speech Synthesis in Simultaneous Speech-to-Speech Translation*, arXiv 2110.08214. (Tin nhắn trước tôi gọi nhầm bài này là "Incremental Speech Synthesis For Speech-To-Speech Translation"; tên đúng là tên vừa nêu.)

Những gì bài nói, theo mục:

- **Mục 3.2 và Hình 2:** độ trễ phải tính đến **thời điểm câu phát xong**, không chỉ lúc phát âm đầu tiên. Âm thanh kéo dài làm tắc nghẽn các câu sau. Thuật toán 1 tính độ trễ từ thời điểm có chữ, thời gian tính toán và thời lượng phát.
- **Mục 4.4, Bảng 2 (TTS tiếng Anh, trước khi tăng cường dữ liệu):** "không nhìn trước" (dòng 4) tệ hơn cả về chất lượng (CER 8,2 % so với 5,6 % ở "nhìn trước 1 từ", dòng 5) lẫn độ trễ (2,4 s so với 1,4 s). Nguyên nhân là âm thanh bị kéo dài: trung bình 7,7 s so với 5,9 s, trong khi bản gốc dài 6,0 s.
- **Mục 4.5, Bảng 3:** bỏ "nhìn trước 1 từ" giảm độ trễ 0,2 đến 0,5 s nhưng mất khoảng 1,0 đến 1,5 BLEU. Thêm "nhìn trước giả" sinh từ bộ dịch thì chất lượng gần bằng nhìn trước thật.
- Hai kỹ thuật chính của bài (TTS được huấn luyện với tăng cường tiền tố, và bộ dịch sinh "nhìn trước giả") **đều cần huấn luyện** và dành cho dịch đồng thời. Ta không dùng được trực tiếp. Dữ liệu là es↔en.

Phần dùng được cho ta, không train:

- **Độ trễ phải đo gồm thời lượng phát** (cơ chế ở Mục 3.2). Nếu ban tổ chức tính đến lúc câu dịch phát xong, tăng tốc độ nói của TTS có thể giảm độ trễ. Cái giá là độ rõ lời; phải kiểm bằng độ đo "heard" (ASR của audio đầu ra). **[Giả thuyết]** với các TTS của ta.
- **Chia câu theo mệnh đề rồi phát mệnh đề đầu trong lúc tổng hợp phần sau.** MT của ta giải mã xong cả câu mới tổng hợp, nên chỉ chia được **sau** khi có câu dịch. Bảng 2 cho thấy TTS chưa được huấn luyện cho đầu vào cụt sẽ kém hơn; cắt ở dấu câu an toàn hơn cắt giữa mệnh đề. TTS của ta đã có cơ chế chia cửa sổ thích ứng (phiên 20/10).

Câu hỏi mở. Ban tổ chức đo độ trễ kiểu nào (đến âm đầu tiên hay đến hết câu)? Tôi chưa thấy điều này trong quy chế đã đọc.

1.7 ASR gia tăng bằng LocalAgreement trên mô hình ASR offline

Kỹ thuật. Chạy lại ASR trên bộ đệm âm thanh đang dài ra, chỉ "chốt" phần chữ giống nhau ở hai lần chạy liên tiếp (LocalAgreement-2). Không cần mô hình streaming.

Tài liệu.

- **[Đã đọc]** Macháček, Dabre, Bojar, *Turning Whisper into Real-Time Transcription System* (2023), arXiv 2307.14743.
- Mục 2 (đoạn "Streaming") và Hình 1: LocalAgreement-2 lấy tiền tố chung dài nhất ở phần chưa chốt của hai lần cập nhật liên tiếp; phần đã chốt không bị sửa lại.
- Mục 2: áp dụng được cho mô hình khác nếu mô hình trả về **mốc thời gian theo từ và dấu câu**.
- Mục 3: cắt bộ đệm khi gặp dấu kết câu, bộ đệm tối đa khoảng 30 s.
- Bảng 1: WER 5,0 đến 5,8 % và độ trễ trung bình 2,85 đến 3,96 s trên GPU A40/L40 với Whisper large-v2. **Đây không phải độ trễ thấp.**

- **[Đã đọc]** Papi, Gaido, Negri, Turchi, *Does Simultaneous Speech Translation need Simultaneous Models?* (2022), arXiv 2204.03783. Tóm tắt và Mục 5: một mô hình huấn luyện offline dùng với chiến lược đồng thời (wait-k), **không train thêm**, cho BLEU cao hơn mô hình huấn luyện đồng thời từ 1 đến 7 BLEU (en→de) và 1 đến 6 BLEU (en→es). Độ trễ trong thí nghiệm khoảng 1,7 đến 3,0 s (Mục 5 và chú thích định nghĩa vùng trễ: thấp dưới 1 s, vừa 1 đến 2 s, cao trên 2 s). Mục 8: độ trễ có tính thời gian tính toán khác xa độ trễ lý tưởng. Đây là dịch giọng nói đầu-cuối en→de/es, không phải cascade.

Của ta. [Giả thuyết] Zipformer-CTC cho mốc thời gian theo khung nên có thể đáp ứng điều kiện. Một lần ASR trên chip tốn khoảng 170 ms (cửa sổ cố định), nên chạy lại mỗi 0,5 đến 1 s khả thi về tính toán nhưng chiếm NPU song song với MT và TTS. Lợi ích thật chỉ có với **câu dài** (nói liên tục trên khoảng 4 s). Với clip ngắn của BTC thì gần như không lợi, cộng rủi ro dịch từng phần (NLLB chưa được huấn luyện cho đầu vào cụt). Ưu tiên thấp. Chỉ làm sau khi đo phân bố độ dài clip và biết cách ban tổ chức đo trễ.

1.8 VAD và điểm kết thúc câu

Tài liệu. [Đã đọc] README của Silero VAD, mục Key Features: mô hình khoảng 2 MB, một khối âm thanh từ 30 ms xử lý dưới 1 ms trên một luồng CPU (README không nêu loại CPU), hỗ trợ 8 và 16 kHz, có bản ONNX. README **không nêu** các tham số `min_silence_duration_ms`, `speech_pad_ms`, `threshold`, và dẫn sang một trang so sánh chất lượng mà tôi chưa đọc.

Của ta. VAD hiện là thuật toán mức năng lượng cục bộ (giới hạn 12 s mỗi đoạn). Thời gian im lặng tối thiểu trước khi cắt là núm điều chỉnh độ trễ lớn nhất của khâu này: giảm thì cắt sớm hơn nhưng dễ cắt giữa câu. **[Giả thuyết]** VAD mô hình như Silero chắc hơn VAD năng lượng trong môi trường ồn; tôi **chưa có số liệu** cho điều này, cần đo trên BTC trộn tiếng ồn (mục 2.5).

1.9 Những thứ đã cân nhắc và để sau

- **Speculative decoding và mô hình nháp.** Chỉ có kết quả tìm kiếm **[Chỉ tóm tắt]**: Decoding Speculative Decoding (2402.01528) nói độ trễ của mô hình nháp là nút thắt; gói nllw là bản thử nghiệm dùng NLLB 600M làm nháp cho 1.3B, README tự nói chưa kiểm chứng. Không có phép đo nào cho cặp NLLB. Bước decoder của ta chỉ 16 ms, và 1.3B đã bị loại ở phiên trước. Để sau.
- **Mô hình giải mã nông, giải mã không tự hồi quy.** Cần huấn luyện lại, ngoài phạm vi.

1.10 Bảng ưu tiên Phần 1

#	Kỹ thuật	Lợi ích kỳ vọng	Chi phí	Kiểm trên laptop?	Ưu tiên
1.1	Cắt từ vựng đầu ra sâu hơn	giảm dung lượng, có thể giảm thời gian bước	thấp (có sẵn script parity)	Có (chrF với logits bị giới hạn)	Cao
1.4	Xác nhận chế độ BURST trên máy	chưa biết	rất thấp	Không (cần máy)	Cao, làm đầu tiên khi có máy
1.3	Chia sẻ trọng số	tới khoảng 0,9 GB	phụ thuộc AI Hub cho phép hay không	Không	Trung bình
1.2	Lượng tử hóa trọng số NLLB	nhỏ hơn, có thể nhanh hơn	trung bình đến cao	Một phần (QuantSim)	Chỉ khi dung lượng là vấn đề
1.5	Đồ thị nhiều kích thước	nhanh hơn cho câu ngắn	trung bình	Một phần (phân bố độ dài)	Trung bình

#	Kỹ thuật	Lợi ích kỳ vọng	Chi phí	Kiểm trên laptop?	Ưu tiên
1.6	Tốc độ nói và phát sớm	giảm độ trễ cảm nhận	thấp	Có (đo "heard")	Phụ thuộc cách đo trễ
1.7	LocalAgreement	chỉ với câu dài	cao, rủi ro	Có	Thấp
1.8	VAD mô hình	chưa biết	thấp	Có	Trung bình

PHẦN 2. Nâng chất lượng mà không huấn luyện lại (hoặc fine-tune nhẹ)

Điểm yếu đã đo: MT cho vi↔ko và vi→zh, WER en→vi 17,8 %. Tiếng ồn doanh nghiệp là yếu tố chưa đo được vì BTC không có điều kiện ồn.

2.1 Làm sạch văn bản giữa ASR và MT (dấu câu, viết hoa, số)

Kỹ thuật. ASR tiếng Việt của ta xuất chữ thường, không dấu câu. NLLB được huấn luyện trên văn bản có dấu câu. Khôi phục dấu câu trước khi dịch cải thiện bản dịch.

Bằng chứng.

- **[Đo của ta]** `vi_punct` (luật khôi phục dấu "?"): vi→ko trên BTC chrF 24,06 → 25,68, BLEU 23,82 → 30,04 (tham chiếu với văn bản gốc chuẩn: 32,97 / 34,61). Bộ luật được chỉnh trên chính bộ BTC nên **có nguy cơ khớp quá mức**.
- **[Đã đọc]** Di Gangi, Enyedi, Brusadin, Federico, *Robust Neural Machine Translation for Clean and Noisy Speech Transcripts* (2019), arXiv 1910.10238, Bảng 3 (en→it, TED, mô hình tổng quát): đầu vào ASR có dấu câu 24,5 BLEU, **không** dấu câu 20,6 BLEU. Mức mất khoảng 4 BLEU trên cặp này.
- **[Chỉ tóm tắt]** Công trình của KIT cho thấy dùng phân đoạn và dấu câu chuẩn của người trên đầu ra ASR tăng 1,78 BLEU; Peitz và cộng sự báo cáo tới 0,8 BLEU từ các phương pháp dự đoán dấu câu (isl.iar.kit.edu). Con số thật phụ thuộc cặp ngôn ngữ và hệ thống.

Việc cần làm.

- Đánh giá `vi_punct` trên **tập giữ riêng** không dùng khi chỉnh luật (đã có quy tắc holdout kín).
- Mở rộng sang dấu phẩy và dấu chấm trong câu dài (hiện chỉ xử lý "?"). Mô hình dấu câu nhỏ là hướng huấn luyện nhẹ.
- Với en/zh/ko, `step1.md` ghi SenseVoice có ITN và dấu câu sẵn (`rich_transcription_postprocess`); chưa kiểm xem pipeline hiện tại có dùng.

2.2 Cải thiện nhận dạng từ chuyên ngành và tên riêng (không train)

Kỹ thuật. ASR CTC cho xác suất theo khung. Có thể dò các từ trong danh sách thuật ngữ (từ doanh nghiệp, tên riêng) bằng đồ thị ngữ cảnh rồi thay vào chỗ kết quả tham lam.

Tài liệu. [Đã đọc] Andrusenko và cộng sự (NVIDIA), *Fast Context-Biasing for CTC and Transducer ASR models with CTC-based Word Spotter* (2024), arXiv 2406.07096.

- Mục 2.1: xây đồ thị ngữ cảnh (trie ghép với cấu trúc chuyển CTC) cho các từ cần ưu tiên, chạy trên log-xác suất CTC, thưởng cho hypothesis đi qua token không-trống (`cbw`); cắt tỉa bằng ngưỡng beam và bỏ qua khung trống.
- **Không sửa mô hình ASR và không train lại** (phần Giới thiệu phân biệt nhóm "shallow fusion", chỉ sửa giải mã).
- Bảng 1 (tập kiểm tra GTC, kết quả CTC): tham lam WER 14,02 %, F-score từ ngữ cảnh 0,32; pyctcdecode bật ngữ cảnh WER 12,06 %, mất 179 s; CTC-WS WER 10,48 %, F-score 0,87, mất 15 s (thời gian giải mã không gồm encoder).
- Cùng bảng: pyctcdecode **không bật** ngữ cảnh (beam search thường) WER 14,17 %, tức không tốt hơn tham lam.
- **Giới hạn.** Bài thử trên tiếng Anh với mô hình NeMo và tập GTC. Cài đặt có sẵn trong NeMo; Zipformer của ta cần mã riêng (xuất log-xác suất CTC, tách token theo từ vựng Zipformer). Danh sách từ ngữ cảnh phải đến từ miền doanh nghiệp, **không** lấy từ bộ kiểm tra BTC.

2.3 Mô hình ngôn ngữ n-gram khi giải mã CTC

Kỹ thuật. Thêm điểm của mô hình n-gram (huấn luyện trên văn bản thuần, không cần âm thanh) vào beam search CTC.

Tài liệu. [Chỉ tóm tắt] Tài liệu NeMo N-gram Language Model Fusion mô tả huấn luyện KenLM từ văn bản và dùng trong beam search với trọng số `beam_alpha`, `beam_beta`; pyctcdecode cung cấp bộ giải mã. Tôi chỉ thấy phần tóm tắt và **không tìm thấy phép đo trực tiếp** (CTC cộng KenLM cộng văn bản miền) với WER trước-sau.

Nhận xét. Bảng 1 của bài CTC-WS (mục 2.2) cho thấy giải mã có ngữ cảnh bằng pyctcdecode chậm gấp khoảng 12 lần CTC-WS trên máy của họ. Giải mã sẽ chạy trên CPU của máy ARM nên thời gian thật phải đo. Nếu chỉ cần ưu tiên danh sách từ, CTC-WS hợp lý hơn.

2.4 Fine-tune nhẹ NLLB (LoRA đã gộp, hoặc adapter) cho miền doanh nghiệp và đầu vào ASR có lỗi

Kỹ thuật. Đóng băng NLLB, chỉ huấn luyện một số ít tham số bổ sung, rồi gộp vào trọng số để không tăng độ trễ.

Tài liệu.

- **[Đã đọc]** Hu và cộng sự, *LoRA: Low-Rank Adaptation of Large Language Models*, arXiv 2106.09685. Tóm tắt: ngang bằng hoặc tốt hơn fine-tune toàn bộ với ít tham số huấn luyện hơn nhiều và, khác adapter, không tăng độ trễ suy luận (ma trận hạng thấp gộp được vào trọng số gốc). Bài không thử trên dịch máy.
- **[Đã đọc]** Bapna, Arivazhagan, Firat, *Simple, Scalable Adaptation for Neural Machine Translation* (2019), arXiv 1909.08478. Lớp adapter nhỏ (công thức 1 đến 3, Mục 3) chèn giữa các lớp của mô hình NMT đã đóng băng. Bảng 1 (en→fr): trên IWSLT'15 adapter đạt 44,63 BLEU so với fine-tune toàn bộ 44,59 (mô hình gốc 41,33); trên JRC-Acquis adapter 63,48 so với 64,13 (gốc 54,60). Adapter **thêm lớp** nên tăng độ trễ nếu không gộp được; LoRA tốt hơn ở điểm này.
- **[Đã đọc]** Di Gangi và cộng sự (xem 2.1), Mục 2 và 7, Bảng 3: fine-tune trên đầu vào ASR nâng 24,5 lên 25,9 BLEU (đầu vào ASR có dấu câu) và 20,6 lên 21,8 (không dấu câu) so với mô hình tổng quát. Kết luận Mục 7: fine-tune trên đầu vào nhiễu cải thiện hơn 2 BLEU trong thiết lập của họ; fine-tune **đồng**

thời trên cả dữ liệu sạch và nhiễu để không quên và không giảm chất lượng trên đầu vào sạch. Mô hình của họ là Transformer Big en→it với 16 triệu cặp câu, khác xa NLLB-600M.

Của ta. Gộp LoRA vào trọng số rồi biên dịch lại (link job AI Hub). Máy huấn luyện là RTX 4050 6 GB: LoRA trên 600M với fp16 khả thi trên lý thuyết, **chưa thử**.

Dữ liệu là điểm khó nhất:

- Cần cặp câu song ngữ vi↔en/zh/ko trong miền doanh nghiệp. Tôi chưa có nguồn cụ thể.
- Cần nguồn "nhiễu ASR": chạy chính ASR của ta lên giọng nói (thật hoặc tổng hợp) để có văn bản lỗi, ghép với câu dịch chuẩn.
- **Không huấn luyện trên bộ BTC mẫu** (100 đến 180 clip): quá nhỏ, nhãn có nhiễu, và dùng nó vừa để huấn luyện vừa để đánh giá làm số đo mất giá trị.
- Phải giữ tập đánh giá riêng và kiểm cả 6 chiều: một bộ LoRA có thể tăng chiều này và làm giảm chiều khác.

2.5 Tiếng ồn: lọc ồn đầu vào, trộn lại tín hiệu gốc, và fine-tune đa điều kiện

Tài liệu và bằng chứng.

- **[Đã đọc]** Iwamoto và cộng sự (NTT), *How bad are artifacts?: Analyzing the impact of speech enhancement errors on ASR* (2022), arXiv 2201.06685.
- Mục 4.2: tín hiệu đã lọc ồn **không cải thiện** ASR so với tín hiệu gốc có ồn (trùng các nghiên cứu trước); nguyên nhân chính là phần "artifact" do chính bộ lọc tạo ra.
- Mục 3.2 (công thức 8) và Mục 4.3 (Hình 3, 4): "observation adding" (OA), cộng một bản **co giãn của tín hiệu gốc** vào tín hiệu đã lọc, hệ số quét từ 0 đến 1,5. Không cần train. Tóm tắt báo cáo OA cải thiện ASR trên dữ liệu mô phỏng lẫn thu thật.
- **Giới hạn.** ASR của bài là Kaldi DNN-HMM huấn luyện trên tiếng ồn và vang; dữ liệu mô phỏng ở SNR 0 dB (WSJ0 cộng ồn CHiME-3), cộng một tập CHiME-3 thu thật. Rất khác Zipformer-CTC. Tôi chưa đọc con số WER cụ thể từ Hình 3 và 4.
- **[Đã đọc]** RescueSpeech (Saarland University, Mila, DFKI; bộ dữ liệu tiếng Đức miền cứu hộ), arXiv 2306.04054, Bảng 2 và 4 (Whisper-large-v2): WER trên đầu vào ồn 50,85 % (mô hình huấn luyện sẵn), 46,70 % sau fine-tune trên dữ liệu sạch, 45,84 % sau fine-tune đa điều kiện (trộn ồn). Ghép lọc ồn và ASR: 48,04 % khi huấn luyện độc lập, 45,29 % khi huấn luyện **chung**.
- **[Đo của ta]** GTCRN (lọc ồn) không cải thiện, WER tăng nhẹ ở SNR 5 dB (phiên trước).

Nhận định.

- Lọc ồn rời rạc, không huấn luyện chung với ASR, thường không giúp. Khớp với kết quả GTCRN của ta.
- **OA là thử nghiệm rẻ nhất:** chỉ là một phép cộng trộn, thử được trên CPU. Không đảm bảo hiệu quả với Zipformer.
- "Fine-tune đa điều kiện" trong RescueSpeech cải thiện khiêm tốn so với fine-tune sạch (khoảng 0,9 điểm WER tuyệt đối trong thí nghiệm đó). Chỉ đáng làm nếu có dữ liệu ồn đúng miền.

Việc cần làm: xây bộ đánh giá ồn. Trộn tiếng ồn văn phòng vào clip BTC ở SNR 20, 10 và 5 dB, chạy `eval_btc.py`. Đo (a) WER theo mức SNR cho từng ASR, (b) lọc ồn tắt, bật, và OA với ω từ 0,3 đến 1,0, (c) VAD năng lượng so với Silero. Tập tiếng ồn: cần nguồn có giấy phép rõ ràng; tôi chưa chọn bộ nào.

2.6 Thuật ngữ cố định ở khâu dịch (glossary)

Hiện trạng. Đã có `glossary` thay trước và sau khi dịch.

Tài liệu. [Đã đọc] Post và Vilar, *Fast Lexically Constrained Decoding with Dynamic Beam Allocation for Neural Machine Translation* (2018), arXiv 1804.06609.

- Mục 3 (thuật toán DBA) và Bảng 1: độ phức tạp O(Nk) (N độ dài câu, k beam), không phụ thuộc số ràng buộc C. So với Grid Beam Search của Hokamp và Liu là O(NkC).
- Bảng 2: sau khi cắt BPE, số ràng buộc ở mức token tăng mạnh (trung bình 1,36 đến 5,45 token mỗi câu tùy bộ).
- Đoạn kết quả tốc độ (Bảng 4 trong văn bản, en→de, WMT14): trên V100, DBA với k = 10 mất khoảng 0,6 s mỗi câu bất kể số ràng buộc, chậm khoảng 3 lần giải mã không ràng buộc; BLEU 26,7 so với 25,6 của GBS với beam gốc 1 ở cùng thời gian. Với k = 5 BLEU của DBA còn 23,5; bài ghi rõ phải có điều chỉnh "bank" thì mới chạy được khi số ràng buộc ≥ k.
- Bài không thử trên NLLB hay tiếng Việt/Hàn/Trung.

Của ta. Beam search của ta chạy trên máy chủ nhưng đồ thị decB cố định B = 4. Với k = 4 và nhiều ràng buộc token, bài chỉ ra chất lượng sẽ kém (k = 5 đã tụt 3,2 BLEU so với k = 10). **Kết luận: DBA khó áp dụng với đồ thị B = 4; giữ cách thay thế thuật ngữ hiện có.** Muốn dùng DBA cần đồ thị beam lớn hơn, trả giá độ trễ.

2.7 Chất lượng giọng đầu ra (TTS) và độ đo "heard"

Điểm số dùng ASR của audio đầu ra, nên độ rõ lời của TTS quyết định điểm.

Tài liệu. [Đã đọc] *SupertonicTTS: Towards Highly Efficient and Streamlined Text-to-Speech System*, arXiv 2503.23108, Mục 5.2, Bảng 3 (số bước flow-matching NFE; RTF đo trên RTX 4090 theo mô tả ở các mục khác, bảng không ghi phần cứng):

NFE	RTF	WER	NISQA
4	0,006	11,43	2,623
8	0,011	2,818	3,916
16	0,019	2,679	3,994
32	0,037	2,639	4,033

Bài chọn NFE = 32 cho các đánh giá sau. WER gần như phẳng từ NFE 8 trở lên, NFE 4 thì sụp đổ; thời gian tăng gần tỉ lệ NFE. Bài chỉ đo tiếng Anh, không có CER hay MOS theo NFE.

Của ta. Cần xác định số bước của đồ thị `tts_enko_supertonic_full_v3` đang biên dịch (chưa kiểm). Nếu đang cao hơn 8, hạ xuống 8 đến 16 sẽ giảm thời gian TTS với mất mát nhỏ theo bảng; nhưng bảng chỉ đúng cho tiếng Anh, tiếng Hàn phải đo lại bằng "heard" trên BTC. Bài học sẵn có: nén int8 vocoder làm âm thanh vỡ.

2.8 Chọn trong nhiều ứng viên dịch (MBR)

Tài liệu. [Đã đọc] Freitag, Grangier, Tan, Liang, *High Quality Rather than High Model Probability: Minimum Bayes Risk Decoding with Neural Metrics*, arXiv 2111.09388, Mục 3.1: bài dùng **lấy mẫu không thiên lệch** để tạo tập ứng viên, nêu chi phí lấy mẫu tuyến tính còn chi phí tính tiện ích bậc hai theo kích thước tập; kết quả

tốt nhất dùng độ đo nơ-ron BLEURT. Với ta (4 ứng viên từ beam, không có độ đo nơ-ron trên thiết bị) bài **không cho bằng chứng** về hiệu quả. Có thể thử MBR với chrF trên 4 ứng viên của decB ở máy chủ trên CPU như một thí nghiệm rẻ, nhưng kỳ vọng thấp; không đưa vào kế hoạch.

2.9 Bảng ưu tiên Phần 2

#	Kỹ thuật	Lợi ích kỳ vọng	Cần train?	Chi phí	Ưu tiên
2.1	Kiểm `vi_punct` trên holdout, mở rộng dấu câu	vi→ko, vi→en	Không (luật)	thấp	**Cao**
2.5	Bộ đánh giá ồn và thử OA	chưa biết, vì chưa đo ồn	Không	thấp	**Cao** (để biết mức hỏng)
2.7	Giảm số bước TTS (nếu đang cao)	giảm trễ TTS	Không (biên dịch lại)	trung bình	Trung bình
2.2	CTC-WS cho từ chuyên ngành	từ hiếm và tên riêng	Không	trung bình (mã mới)	Trung bình
2.4	LoRA gộp cho MT	miền và nhiễu ASR	**Có (nhẹ)**	cao (dữ liệu, biên dịch lại)	Trung bình đến thấp
2.3	KenLM cho CTC	chưa có số liệu	Không	trung bình	Thấp
2.6	DBA thay glossary	không khả thi với B = 4	Không	cao	Thấp
2.8	MBR 4 ứng viên	chưa có bằng chứng	Không	thấp	Thấp

PHẦN 3. Kế hoạch thử nghiệm không cần máy thật (2 ngày)

Ngày 1 (chất lượng, trên CPU):

- Dựng công cụ trộn ồn vào BTC (SNR 20, 10, 5 dB) và chạy lại `eval_btc.py`: WER theo SNR; lọc ồn bật hoặc tắt; OA với nhiều hệ số; VAD năng lượng so với Silero (mục 2.5).
- Đánh giá `vi_punct` trên tập giữ riêng (mục 2.1).
- Đếm phân bố độ dài clip BTC để quyết định mục 1.5 và 1.7.

Ngày 2 (độ trễ và dung lượng, mô phỏng):

- Giới hạn logits của decoder theo từ vựng đích trên CPU, so chrF với bản hiện tại; đếm số token mỗi ngôn ngữ (mục 1.1).
- Đo "heard" ở các tốc độ nói TTS khác nhau để thấy đánh đổi độ rõ lời và độ dài (mục 1.6).
- Chạy QuantSim W8A16 cho NLLB nếu muốn giảm dung lượng MT (mục 1.2).

Khi có máy: `recon.sh` → `install.sh` → `session0.sh` → `run_btc.sh`; thêm so BURST với mặc định (mục 1.4) và đo thời gian từng bước decoder để kiểm giả thuyết băng thông (mục 1.1).

Việc chưa biết, cần xác nhận

- Ban tổ chức đo độ trễ kiểu nào (đến âm đầu tiên hay đến hết câu)? Quyết định giá trị của mục 1.6 và 1.7.
- AI Hub có cho bật weight sharing khi link nhiều graph không? Quyết định mục 1.3 và 1.5.
- Đồ thị TTS Supertonic đang biên dịch với bao nhiêu bước? (mục 2.7)
- Băng thông bộ nhớ thực tế của QCS8550 và việc bước decoder có bị giới hạn bởi băng thông không (giả thuyết nền của mục 1.1 và 1.2).
- Chất lượng riêng của encoder SenseVoice w8a16.

Tài liệu tham khảo

Đã đọc đúng đoạn nêu:

- Ushio và cộng sự, Vocabulary Trimming: arXiv 2305.15020
- Macháček và cộng sự, Whisper-Streaming: arXiv 2307.14743
- Papi và cộng sự, mô hình offline cho SimulST: arXiv 2204.03783
- Liu và cộng sự, From Start to Finish (incremental TTS): arXiv 2110.08214
- Post và Vilar, Dynamic Beam Allocation: arXiv 1804.06609
- Andrusenko và cộng sự, CTC-WS: arXiv 2406.07096
- Iwamoto và cộng sự, How bad are artifacts: arXiv 2201.06685
- RescueSpeech: arXiv 2306.04054
- Freitag và cộng sự, MBR với độ đo nơ-ron: arXiv 2111.09388
- Bapna và cộng sự, adapter cho NMT: arXiv 1909.08478
- Hu và cộng sự, LoRA: arXiv 2106.09685
- Di Gangi và cộng sự, robust NMT: arXiv 1910.10238
- SupertonicTTS: arXiv 2503.23108
- AIMET quantization workflow: trang tài liệu
- AI Hub quantize examples: trang tài liệu
- CTranslate2 quantization: trang tài liệu
- QNN HTP backend: trang tài liệu (tôi mới đọc 100.000 trên 227.275 ký tự đầu của trang)
- Silero VAD README: GitHub

Chỉ thấy tóm tắt (không dùng làm căn cứ chính): KIT/Peitz về dấu câu (isl.iar.kit.edu); NeMo N-gram LM fusion; pyctcdecode; Decoding Speculative Decoding (2402.01528); nllw (PyPI); trang Qualcomm Profile your model và QNN Benchmarking.